

Mobile Application Development

Lecture 04

Today's Agenda

- Application Fundamental Component
- Developing First (The legendary “Hello World!”) App

Application Fundamental Components

- Activities
- Some Additional Building Blocks

Activity

- An activity represents a single screen with a user interface and action on the user interaction can be performed with the help of activity.
- For example, an email application might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails.
- If an application has more than one activity, then one of them should be marked as the activity that is presented when the application is launched.
- An activity is implemented as a subclass of Activity class as follows –

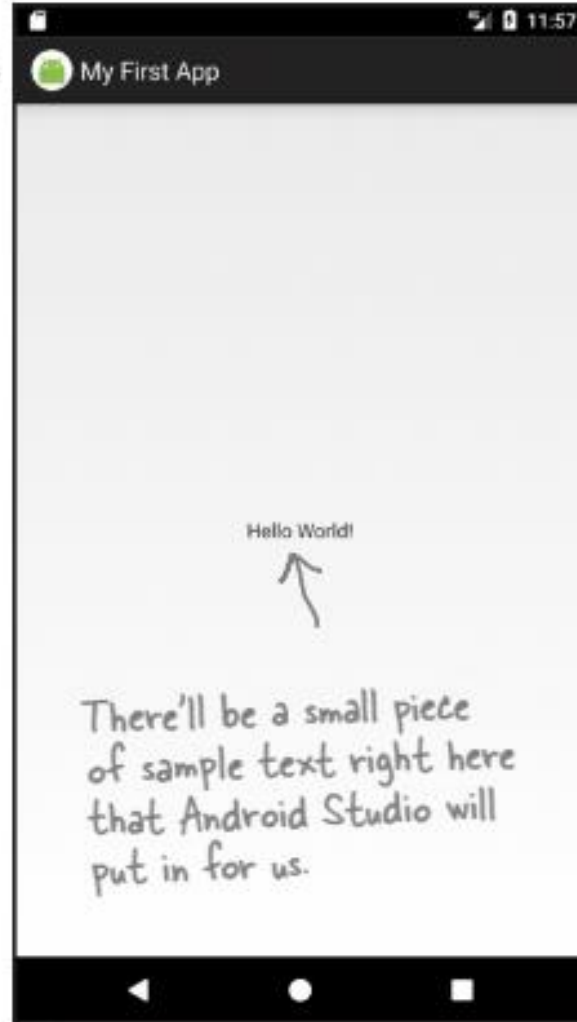
```
public class MainActivity extends Activity {  
    //Todo  
}
```

Some Addition Building blocks for an App

- **Fragments**
 - Represents a portion of user interface in an Activity.
- **Views**
 - UI elements that are drawn on-screen including buttons, lists forms etc.
- **Layouts**
 - A layout define visual structure of a user interface.
- **Intents**
 - Intent allow user to coordinate function of different activity to achieve task.
- **Resources**
 - External elements, such as strings, constants and drawable pictures.
- **Manifest**
 - Configuration file for the application.

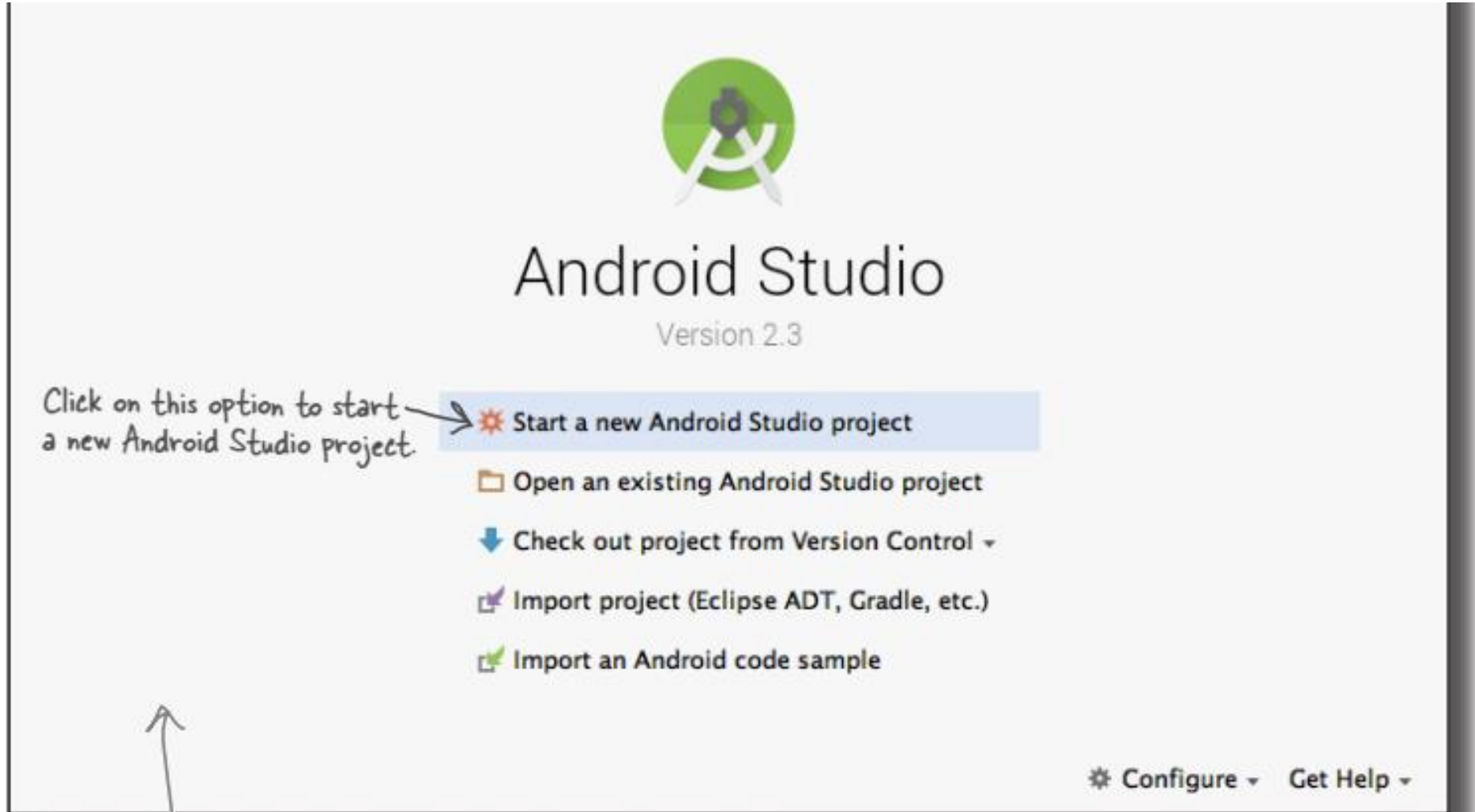
Build a basic app – The legendary “Hello World”

This is the name of the application.



This is a very simple app, but that's all you need for your very first Android app.

Create a new project



Click on this option to start a new Android Studio project.

Start a new Android Studio project

Open an existing Android Studio project

Check out project from Version Control

Import project (Eclipse ADT, Gradle, etc.)

Import an Android code sample

Configure Get Help

Any projects you create will appear here. This is our first project, so this area is currently empty.

Some versions of Android Studio may have an extra option asking if you want to include Kotlin support. Uncheck this option if it's there.

The wizard forms the package name by combining the application name and the company domain.

Create New Project

New Project

Android Studio

Configure your new project

The application name is shown in the Google Play Store and various other places, too.

Application name:

Company domain:

Package name: [Edit](#)

☐ Include C++ support

Use a company domain of hfad.com.

Uncheck the option to include C++ support. If prompted, also uncheck the option to include Kotlin support.

Project location: ...

All of the files for your project will be stored here.

Cancel Previous **Next** Finish

The minimum required SDK is the lowest version your app will support. Your app will run on devices with this level API or higher. It won't run on devices with a lower API.

Create New Project

Target Android Devices

Select the form factors your app will run on

Different platforms may require separate SDKs

☒ Phone and Tablet

Minimum SDK: API 19: Android 4.4 (KitKat)

Lower API levels target more devices, but have fewer features available. By targeting API 19 and later, your app will run on approximately 73.9% of the devices that are active on the Google Play Store.
[Help me choose](#)

☐ Wear

Minimum SDK: API 21: Android 5.0 (Lollipop)

☐ TV

Minimum SDK: API Lollipop: Android 5.0 (Lollipop preview)

☐ Android Auto

☐ Glass

Minimum SDK: Glass Development Kit Preview (API 19)

Cancel Previous **Next** Finish

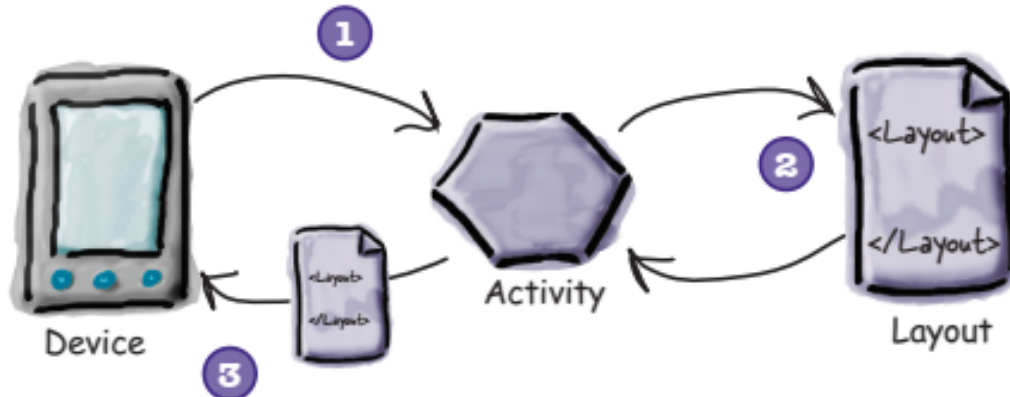
https://en.wikipedia.org/wiki/Android_version_history

Let's see the android versions, codenames and API Level provided by Google.

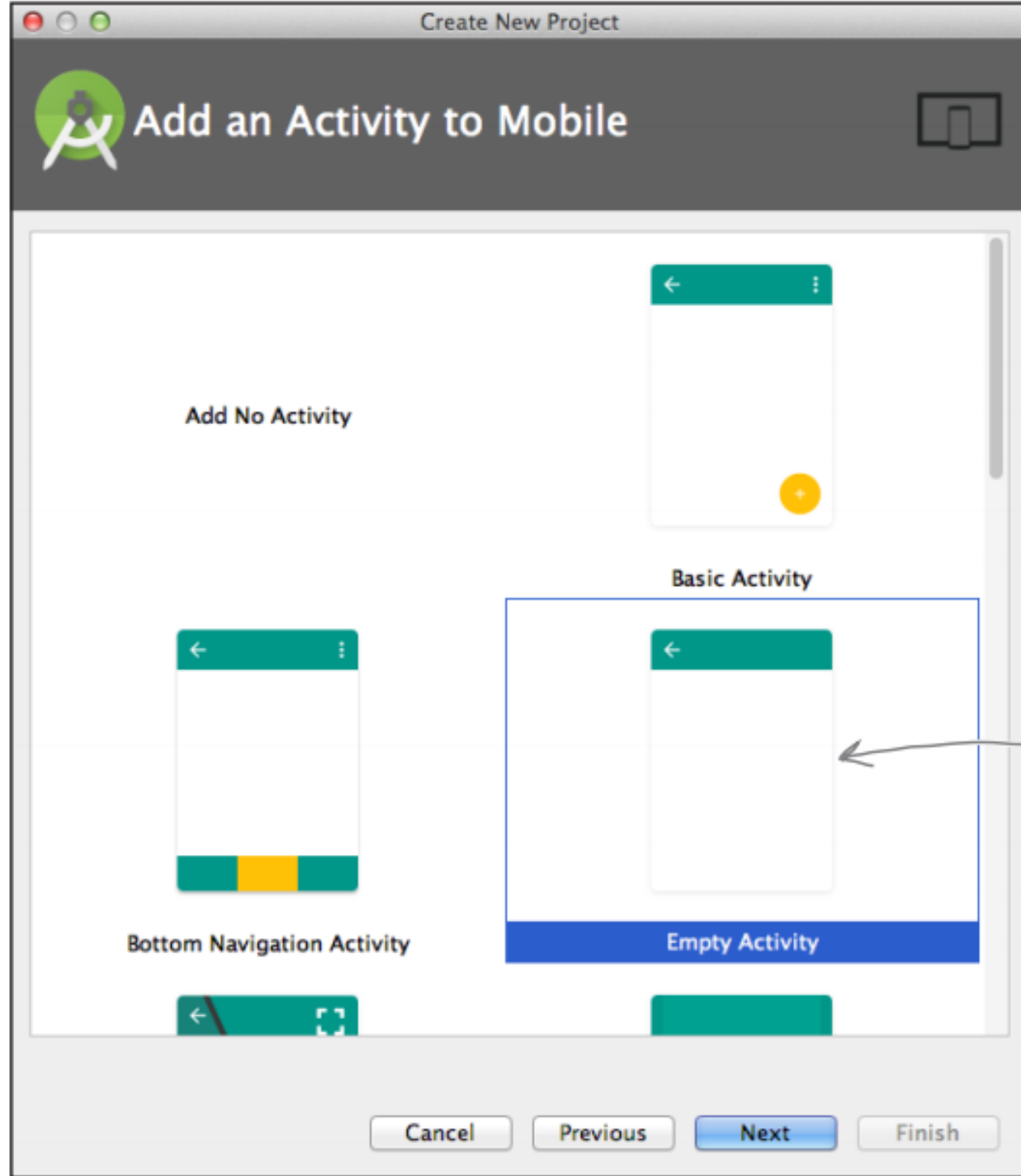
Version	Code name	API Level
1.5	Cupcake	3
1.6	Donut	4
2.1	Eclair	7
2.2	Froyo	8
2.3	Gingerbread	9 and 10
3.1 and 3.3	Honeycomb	12 and 13
4.0	Ice Cream Sandwich	15
4.1, 4.2 and 4.3	Jelly Bean	16, 17 and 18
4.4	KitKat	19
5.0	Lollipop	21
6.0	Marshmallow	23
7.0	Nougat	24-25
8.0	Oreo	26-27
9.0	Pie	28
10.0	Queen Cake	29

Activities and layouts

- 1 The device launches your app and creates an activity object.
- 2 The activity object specifies a layout.
- 3 The activity tells Android to display the layout onscreen.
- 4 The user interacts with the layout that's displayed on the device.
- 5 The activity responds to these interactions by running application code.
- 6 The activity updates the display...
- 7 ...which the user sees on the device.



Add an activity



There are other types of activity you can choose from, but for this exercise make sure you select the Empty Activity option.

Customize the activity

Create New Project

Customize the Activity

Creates a new empty activity

Activity Name:

☒ Generate Layout File

Layout Name:

☐ Backwards Compatibility (AppCompat)

Empty Activity

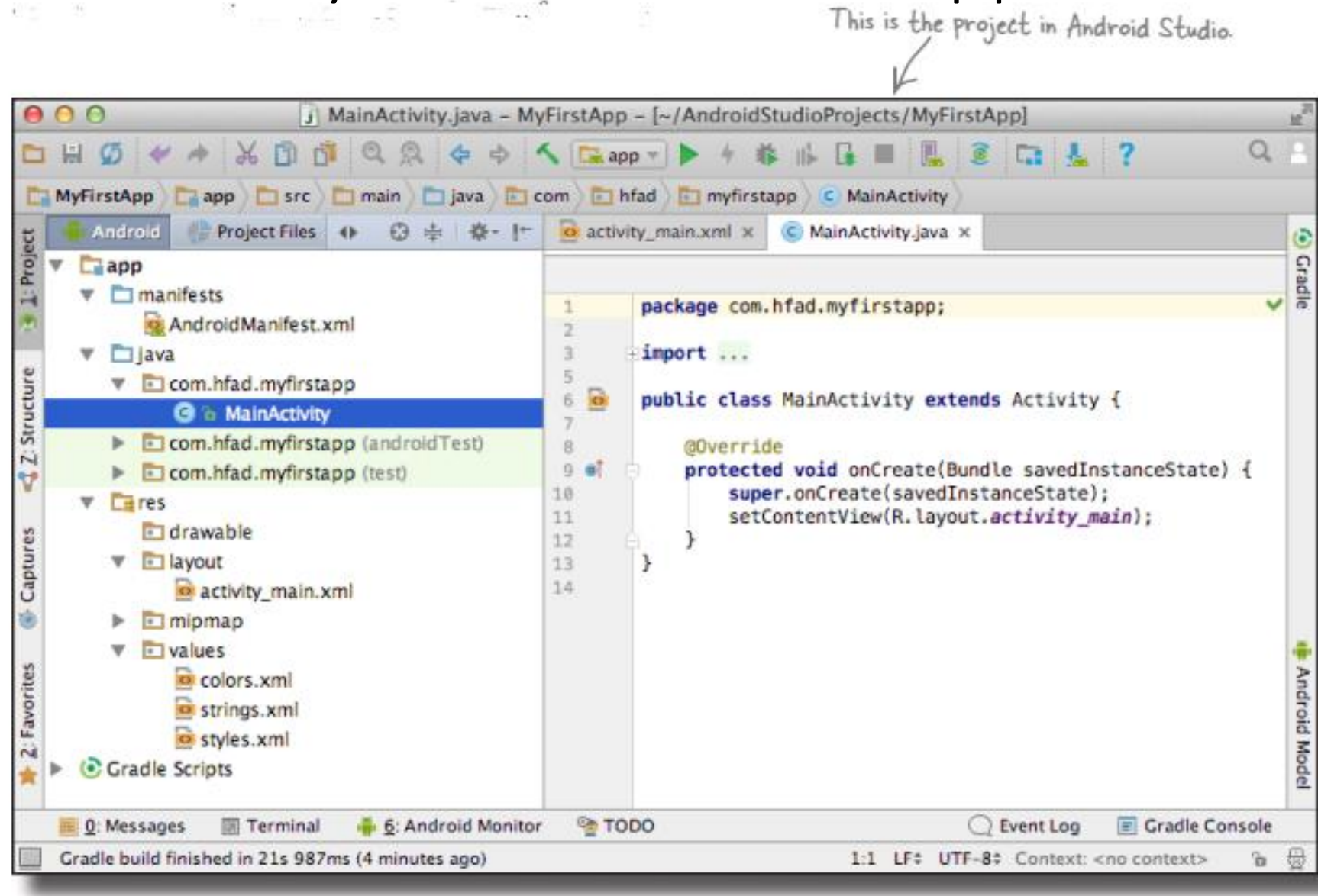
Uncheck the Backwards Compatibility (AppCompat) option. You'll find out more about this setting later

Name the activity "MainActivity" and the layout "activity_main". Also make sure the option to generate the layout is checked.

If false, this activity base class will be Activity instead of AppCompatActivity

Cancel Previous Next Finish

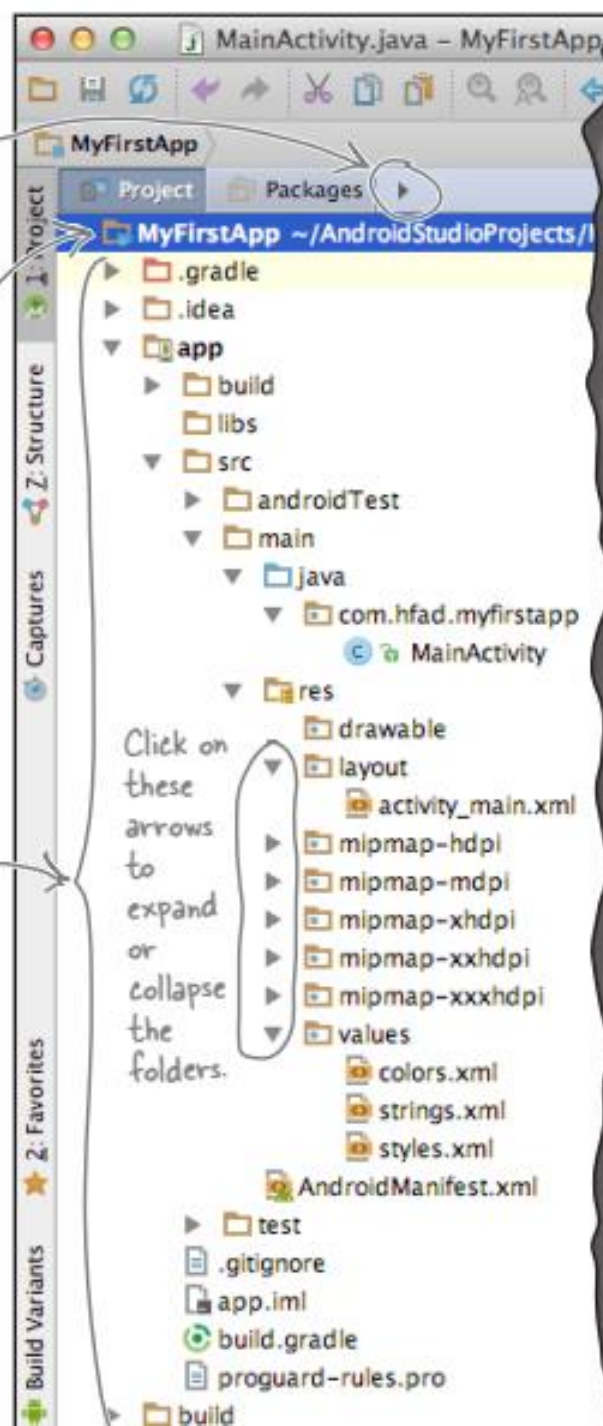
You've just created your first Android app



Click on the arrow here and choose the Project option to see the files and folders that make up your project.

This is the name of the project.

These files and folders are all included in your project.



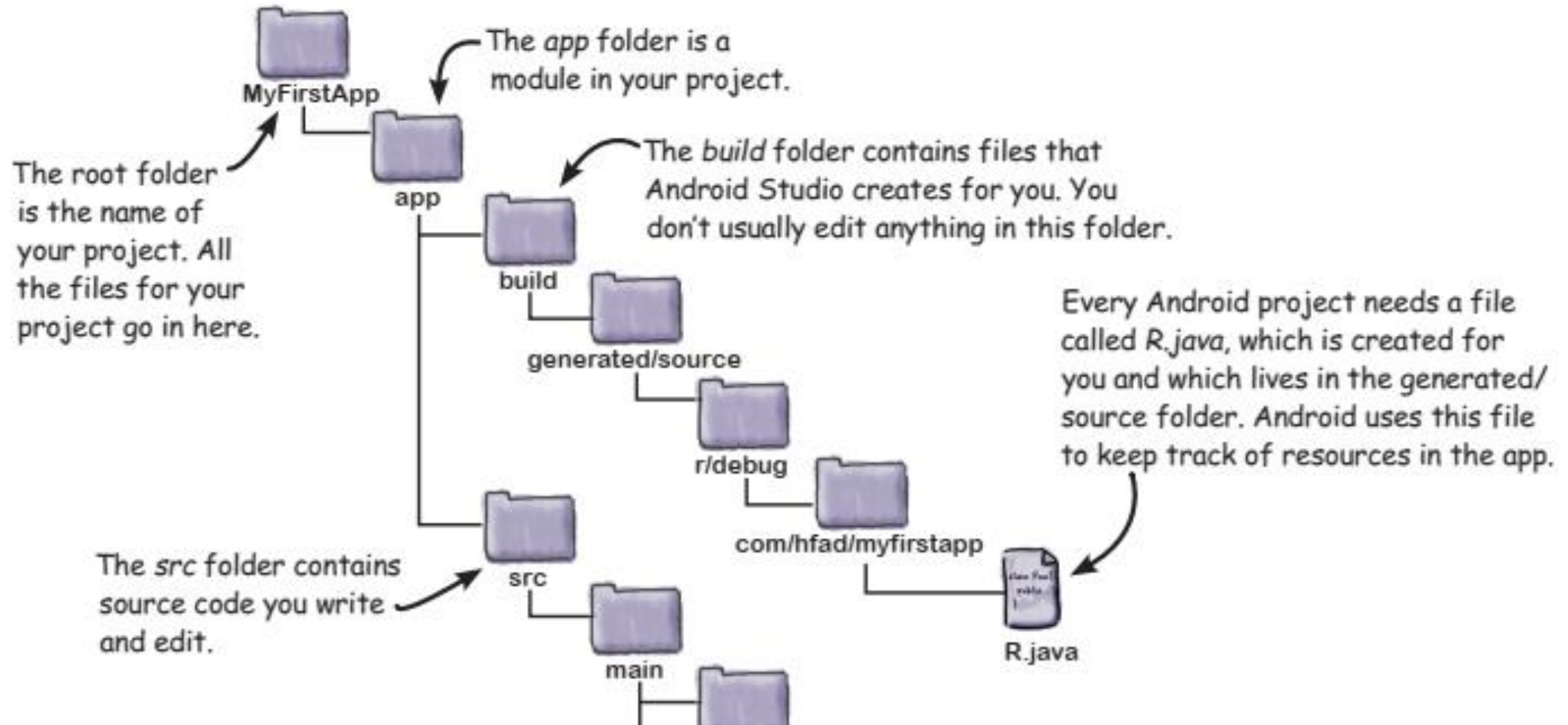
The folder structure includes different types of files

If you browse through the folder structure, you'll see that the wizard has created various types of files and folders for you:

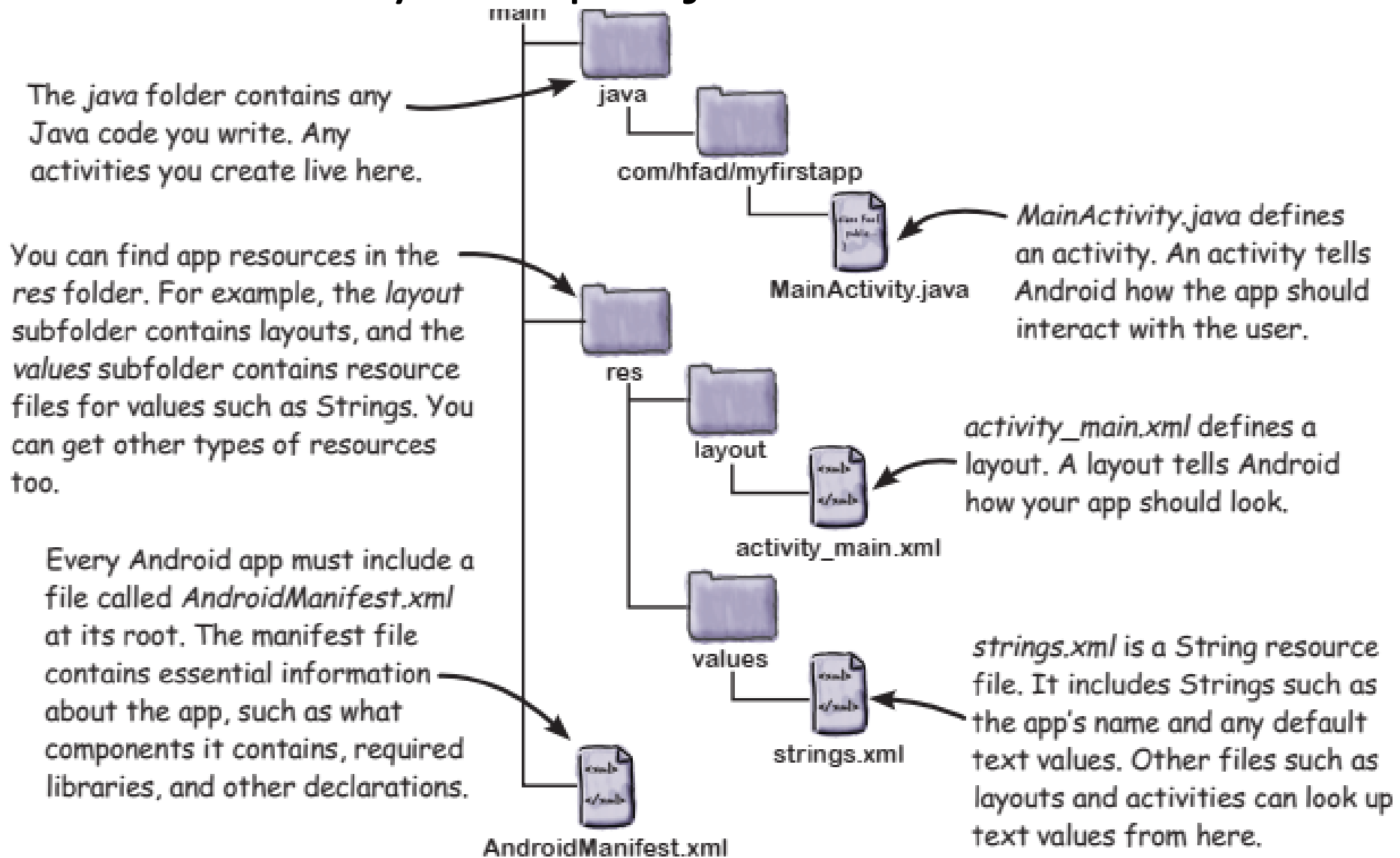
- ★ **Java and XML source files**
These are the activity and layout files for your app.
- ★ **Android-generated Java files**
There are some extra Java files you don't need to touch that Android Studio generates for you automatically.
- ★ **Resource files**
These include default image files for icons, styles your app might use, and any common String values your app might want to look up.
- ★ **Android libraries**
In the wizard, you specified the minimum SDK version you want your app to be compatible with. Android Studio makes sure your app includes the relevant Android libraries for that version.
- ★ **Configuration files**
The configuration files tell Android what's actually in the app and how it should run.

Let's take a closer look at some of the key files and folders in Androidville.

Useful files in your project



Useful files in your project

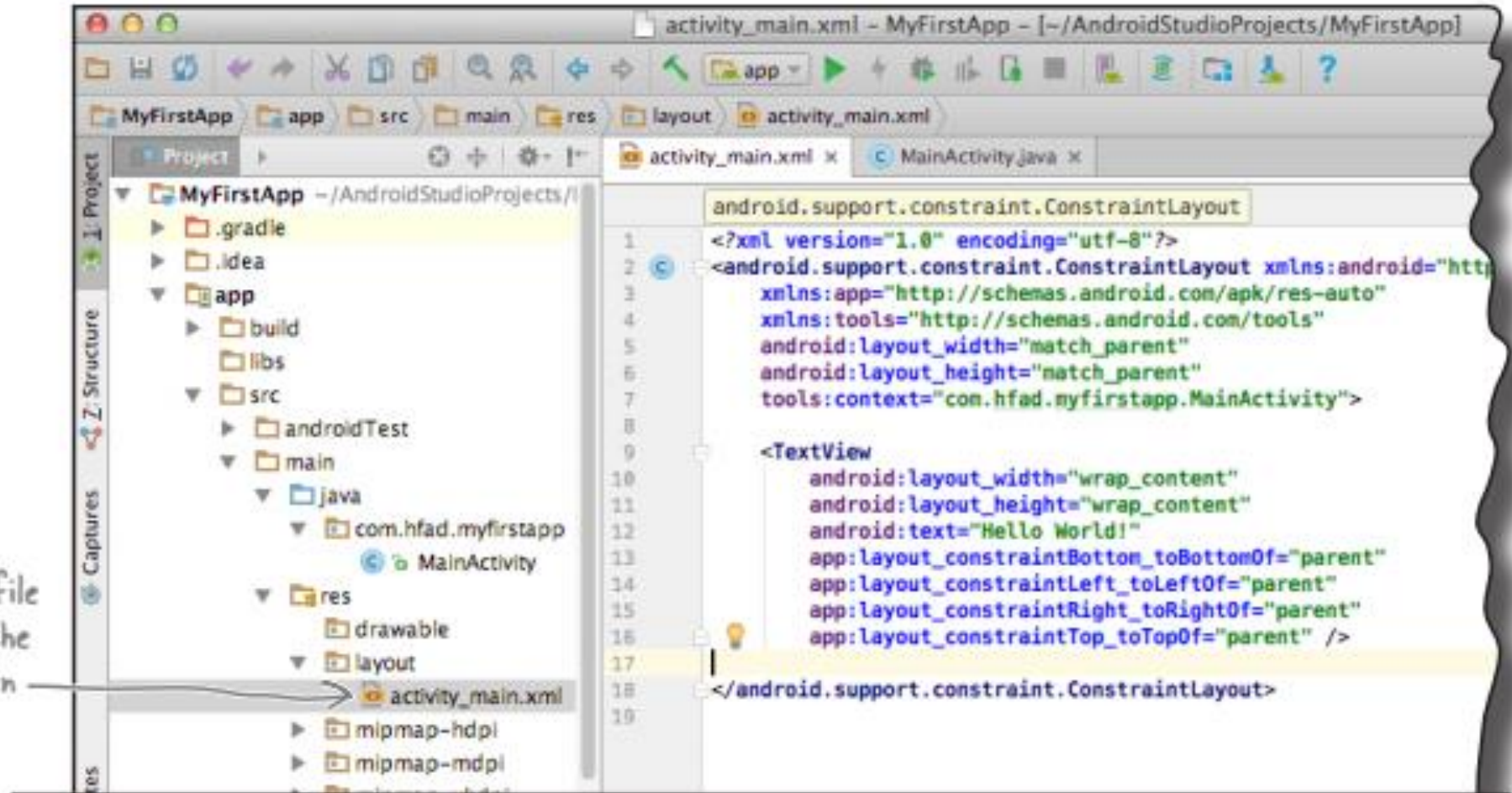


Edit code with the Android Studio editors

The code editor

Most files get displayed in the code editor, which is just like a text editor, but with extra features such as color coding and code checking.

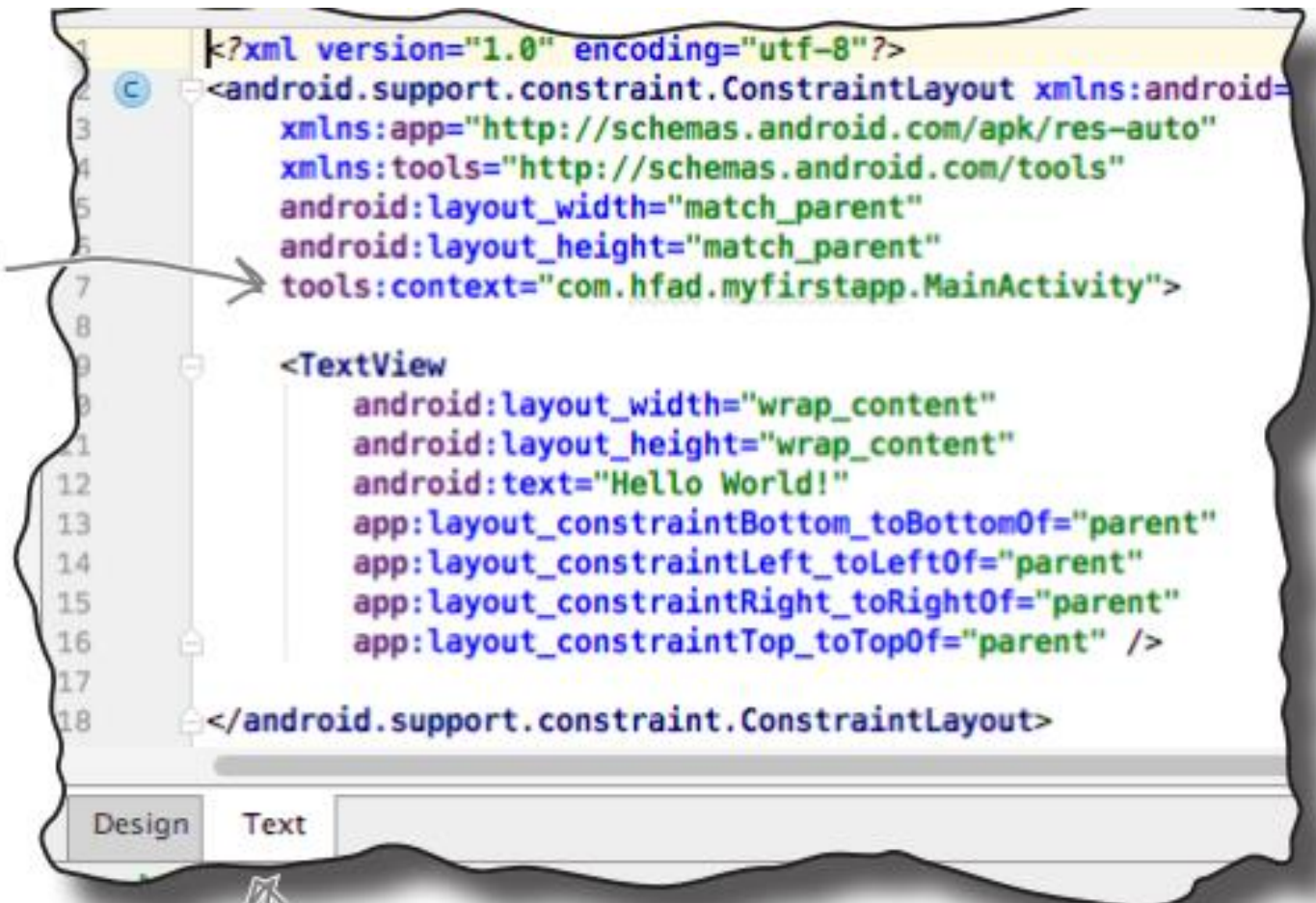
Double-click on the file in the explorer and the file contents appear in the editor panel.



The code editor

When you choose the code editor option, the content of *activity_main.xml* is displayed. Let's take a closer look at it.

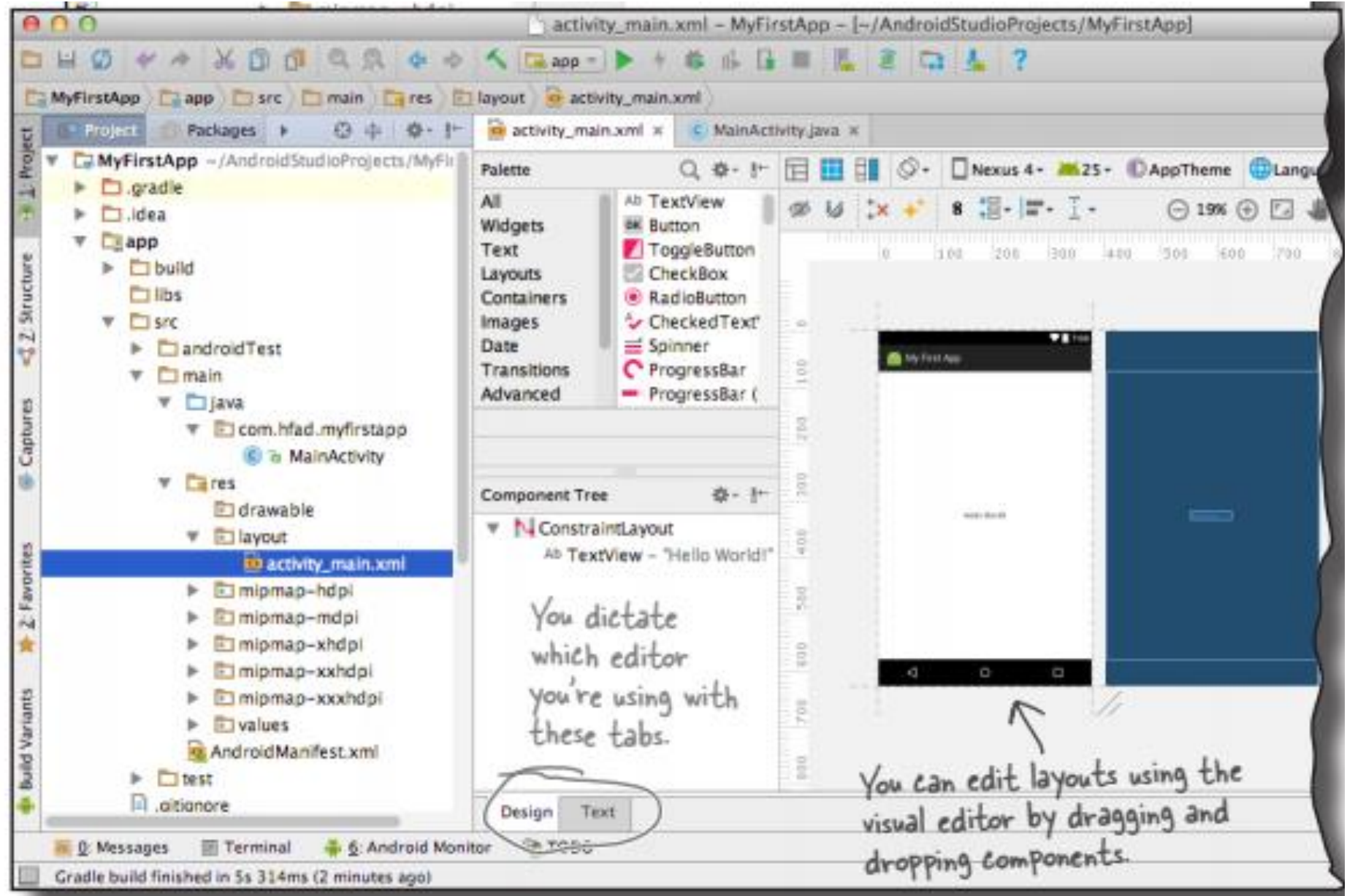
The code editor



To see the code editor, click on "Text" in the bottom tab.

The design editor

If you're editing a layout, you have an extra option. Rather than edit the XML (such as that shown on the next page), you can use the design editor, which allows you to drag GUI components onto your layout, and arrange them how you want. The code editor and design editor give different views of the same file, so you can switch back and forth between the two.



```
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingLeft="16dp"
android:paddingRight="16dp"
android:paddingTop="16dp"
android:paddingBottom="16dp"
android:orientation="vertical"
tools:context="com.hfad.myfirstapp.MainActivity">
```

```
<TextView
    {
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!" />
</LinearLayout>
```

Add padding to the screen margins.

Include a <TextView> GUI component for displaying text.

Make the GUI component just large enough for its content.

Display the String "Hello World!"

Make the layout the same width and height as the screen size on the device.

MainActivity.java

```
package com.hfad.myfirstapp;
```

```
import android.os.Bundle;
```

```
import android.app.Activity;
```

```
public class MainActivity extends Activity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_main);
```

```
    }
```

```
}
```

This is the package name.

These are Android classes
used in MainActivity.

Specify which layout to use.

Implement the `onCreate()`
method from the `Activity`
class. This method is called
when the activity is first
created.

MainActivity extends the
Android class
`android.app.Activity`.

The activity controls what the app does

Android Studio created an activity for us called *MainActivity.java*. The activity specifies what the app **does** and how it should respond to the user.

The layout controls the app's appearance

MainActivity.java specifies that it uses the layout Android Studio created for us called *activity_main.xml*. The layout specifies what the app **looks like**.

